Digital Interface Accessibility Adviser

Expert System Assignment Report

Student name	_____
Student ID	_____
Module	Expert Systems
Submission date	_____

A rule-based adviser for preliminary accessibility assessment of web, desktop, and mobile software interfaces.

Abstract

This project develops a rule-based expert system that supports a preliminary accessibility review of websites, web applications, desktop applications, and mobile applications. A reviewer manually inspects one interface and answers guided questions. The system converts those answers into working-memory facts, determines which checks apply, and uses 20 explicit IF-THEN rules to identify possible accessibility issues. It recommends corrective actions and explains the rules and evidence used. The knowledge base contains 50 domain facts derived mainly from WCAG 2.2 and WCAG2ICT. The implementation uses SWI-Prolog, a local browser interface, forward chaining, backward chaining, and an 18-test automated test suite. All automated tests passed. The adviser is intended for learning and preliminary review; it does not claim complete accessibility conformance.

1. Introduction

Digital interfaces are used for education, employment, communication, banking, health services, and public services. An interface can create barriers when information is available only visually, controls cannot be operated using supported input methods, labels are unclear, focus is invisible, or errors are poorly explained. Accessibility aims to ensure that people with visual, auditory, physical, speech, cognitive, and neurological disabilities can perceive information and operate software [1].

Accessibility evaluation involves both objective requirements and human judgement. Automated tools can detect some problems, but a knowledgeable person is still needed for many checks [1]. This makes accessibility review suitable for a small expert system: reliable domain knowledge can be represented as facts and rules, while the user supplies observations that require human inspection.

2. Domain Selection and Understanding

2.1 Selected domain

The selected domain is **accessibility assessment of digital software interfaces**. The system covers:

- websites and web applications;
- desktop applications; and
- mobile applications.

Physical controls and hardware accessibility are outside the scope. The system reviews one interface at a time against 20 selected Level A and Level AA criteria.

2.2 System specialization

The Digital Interface Accessibility Adviser specializes in converting manual observations into preliminary accessibility conclusions. It knows:

- selected accessibility requirements;
- which interface features make a requirement applicable;
- extra applicability conditions for non-web software;
- what observation indicates a possible problem; and
- what action can be recommended for each possible problem.

2.3 Problem solved

Developers and beginner reviewers may overlook barriers or may not know which requirement applies. Reading a complete standard during every initial review is difficult. The system provides a consistent question sequence and makes its reasoning visible.

2.4 Decisions and recommendations

For each selected criterion, the system produces one of four conclusions:

Conclusion	Meaning
Possible issue	The criterion applies and the reviewer reported a failing observation.
No issue reported	The criterion applies and the reviewer answered Yes to the selected check.
Not applicable	A required interface feature or platform capability is absent.
Needs review	The interface type, a required feature, or the check answer is unknown.

For a possible issue, the adviser gives a corrective recommendation, identifies the rule that fired, displays the supporting facts, and links to the relevant W3C document.

2.5 Domain study

The main domain source is WCAG 2.2, a W3C Recommendation containing testable, technology-independent accessibility success criteria [2]. WCAG was designed for web content. Therefore, WCAG2ICT was also studied. WCAG2ICT explains how WCAG criteria can be interpreted for non-web documents and software, including mobile and desktop applications [3]. For example, it explains that native keyboard checks depend on a platform keyboard interface and that native window or screen titles depend on platform title support.

Expert system and Prolog concepts were studied using established AI and Prolog texts. Russell and Norvig describe knowledge representation and logical inference as central AI topics [4]. Bratko presents Prolog as a practical language for AI and includes knowledge representation and expert systems [5]. The SWI-Prolog documentation was used for the implementation and test framework [6]. A domain-expert interview was not required; knowledge was acquired from these reliable external sources.

3. Knowledge Engineering

3.1 Process followed

The project followed this process:

Domain Selection -> Domain Study -> Knowledge Acquisition -> Knowledge Representation -> Facts and Rules -> Inference -> Testing -> Explanation

1. **Domain selection:** Digital interface accessibility was selected because it is clearly bounded, internationally relevant, and suitable for rule-based advice.
2. **Domain study:** WCAG 2.2, WCAG2ICT, expert-system texts, Prolog material, and SWI-Prolog documentation were reviewed.
3. **Knowledge acquisition:** Twenty criteria were chosen to cover content, presentation, interaction, navigation, language, and forms.
4. **Knowledge representation:** Criterion details, interface features, thresholds, interface types, and native-software conditions were represented as Prolog facts.
5. **Facts and rules:** Each selected criterion was paired with one explicit issue rule.
6. **Inference:** Forward chaining derives applicability, issues, and recommendations. Backward chaining proves a selected issue goal.
7. **Testing:** Individual rules, combined scenarios, exceptions, unknown answers, and explanations were tested.
8. **Explanation:** Each conclusion includes its status, evidence, rule identifier, recommendation, and source.

3.2 Knowledge acquisition and conversion

Each source criterion was converted using the same method:

1. Read the official criterion and relevant interpretation guidance.

2. Identify the interface feature that makes the criterion relevant.
3. Convert the requirement into a Yes/No/Not sure question.
4. Treat a verified No answer as evidence of a possible issue.
5. Write an IF-THEN rule that requires both applicability and the failing observation.
6. Add a short corrective action and the official source link.

Example conversion:

- Source knowledge: WCAG 2.1.1 requires applicable functionality to be operable through a keyboard interface [2]. WCAG2ICT states that, for non-web software, this applies where the platform provides a device-independent keyboard interface service [3].
- Facts: the interface type is desktop; interactive controls are present; a platform keyboard service exists; keyboard operation was reported as No.
- Rule: IF the keyboard check applies AND keyboard operation is No, THEN flag rule r09.
- Recommendation: provide keyboard operation for every applicable function.

3.3 Knowledge representation

The knowledge base uses five fact structures:

```
criterion(Key, Number, Title, Level, Group, RequiredFeatures,
          AnswerKey, Question, Help, SuggestedAction, SourceFragment).

feature(Key, Label, Description).
threshold(Name, Value).
interface_type(Type, Label).
extra_gate(Criterion, Context, Feature).
```

User answers become working-memory observations such as:

```
obs(interface_type, desktop).
obs(interactive_controls, yes).
obs(keyboard_service, yes).
obs(keyboard_operable, no).
```

This representation separates stable domain knowledge from the observations supplied for one review.

3.4 Fact inventory

The knowledge base contains 50 domain facts:

Fact type	Count	Purpose
Criterion facts	20	Store the selected requirement, question, action, and source.
Feature facts	17	Describe interface features and platform capabilities.
Threshold facts	6	Store contrast, resize, and reflow values.
Interface-type facts	3	Represent web, desktop, and mobile interfaces.
Native applicability facts	4	Add keyboard and title conditions to native software.
Total	**50**	Exceeds the required 20 facts.

The 20 main criterion facts are:

Fact ID	Criterion	Level	Knowledge represented
c111	1.1.1 Non-text Content	A	Informative non-text content needs an equivalent text alternative.
c131	1.3.1 Info and Relationships	A	Visual structure and relationships must be available programmatically or in text.

Fact ID	Criterion	Level	Knowledge represented
c133	1.3.3 Sensory Characteristics	A	Instructions must not rely only on sensory characteristics.
c141	1.4.1 Use of Color	A	Colour must not be the only visual method of conveying information.
c143	1.4.3 Contrast (Minimum)	AA	Applicable text must meet minimum contrast.
c144	1.4.4 Resize Text	AA	Text resizing must not cause loss of content or function.
c1410	1.4.10 Reflow	AA	Applicable content must reflow at the required or supported size.
c1411	1.4.11 Non-text Contrast	AA	Required visual controls, states, and graphics need sufficient contrast.
c211	2.1.1 Keyboard	A	Applicable functions must be keyboard-operable.
c212	2.1.2 No Keyboard Trap	A	Keyboard focus must be able to leave a component.
c242	2.4.2 Page or View Titled	A	A page, window, or screen needs a descriptive programmatic title.
c243	2.4.3 Focus Order	A	Sequential focus order must preserve meaning and operation.
c244	2.4.4 Link Purpose	A	The purpose of hyperlink-style controls must be clear.
c246	2.4.6 Headings and Labels	AA	Headings and labels must describe topic or purpose.
c247	2.4.7 Focus Visible	AA	Keyboard focus must be visible.
c253	2.5.3 Label in Name	A	An accessible name must contain the visible label text.
c311	3.1.1 Interface Language	A	The default human language must be programmatically determinable.
c331	3.3.1 Error Identification	A	Detected errors must identify the item and describe the problem in text.
c332	3.3.2 Labels or Instructions	A	Required user input needs labels or instructions.
c333	3.3.3 Error Suggestion	AA	A known and safe correction should be suggested.

3.5 Rule inventory

The inference engine contains 20 explicit issue rules:

Rule	IF conditions	THEN conclusion
r01	c111 applies AND alt_text = no	Flag possible Non-text Content issue.
r02	c131 applies AND programmatic_structure = no	Flag possible Info and Relationships issue.
r03	c133 applies AND sensory_independent = no	Flag possible Sensory Characteristics issue.
r04	c141 applies AND colour_not_only_cue = no	Flag possible Use of Color issue.
r05	c143 applies AND text_contrast_ok = no	Flag possible minimum text-contrast issue.
r06	c144 applies AND text_resizes = no	Flag possible Resize Text issue.

Rule	IF conditions	THEN conclusion
r07	c1410 applies AND reflow_ok = no	Flag possible Reflow issue.
r08	c1411 applies AND nontext_contrast_ok = no	Flag possible Non-text Contrast issue.
r09	c211 applies AND keyboard_operable = no	Flag possible Keyboard issue.
r10	c212 applies AND focus_can_leave = no	Flag possible Keyboard Trap issue.
r11	c242 applies AND view_title_descriptive = no	Flag possible Page or View Titled issue.
r12	c243 applies AND focus_order_ok = no	Flag possible Focus Order issue.
r13	c244 applies AND link_purpose_clear = no	Flag possible Link Purpose issue.
r14	c246 applies AND headings_labels_descriptive = no	Flag possible Headings and Labels issue.
r15	c247 applies AND focus_visible = no	Flag possible Focus Visible issue.
r16	c253 applies AND name_contains_label = no	Flag possible Label in Name issue.
r17	c311 applies AND interface_language_exposed = no	Flag possible Interface Language issue.
r18	c331 applies AND errors_identified = no	Flag possible Error Identification issue.
r19	c332 applies AND inputs_labelled = no	Flag possible Labels or Instructions issue.
r20	c333 applies AND correction_suggested = no	Flag possible Error Suggestion issue.

The explicit rules are implemented as `issue_rule/4` clauses. General inference clauses separately derive applicability, non-applicability, and recommendations.

4. System Implementation

4.1 Technology

The system uses SWI-Prolog. Prolog was selected because its facts, rules, unification, list processing, and backtracking support symbolic knowledge representation and inference [5]. SWI-Prolog also provides HTTP and JSON libraries for the local user interface and PlUnit for automated testing [6].

The interface uses plain HTML, CSS, and JavaScript. It has no external JavaScript framework or database. The server is local and does not upload review answers.

4.2 Main components

File	Responsibility
knowledge_base.pl	Domain facts, questions, recommendations, thresholds, and source identifiers.
inference_engine.pl	Twenty issue rules, forward chaining, backward proof, result classification, and explanations.
web_server.pl	Local HTTP server and JSON endpoints.
web/index.html	User-interface structure.
web/app.js	Loads questions, collects answers, calls the API, and renders results.
web/style.css	Simple responsive presentation and visible focus styles.
tests.pl	Automated unit and scenario tests.

4.3 System architecture

```
User inspects an interface
          |
          v
Browser questionnaire
          |
          v  JSON answers
Local SWI-Prolog server
          |
          +--> Knowledge base (50 facts)
          |
          +--> Inference engine (20 issue rules)
          |
          v
Status + recommendation + explanation + sources
```

4.4 User interface

The UI has four stages:

1. Choose website/web app, desktop app, or mobile app.
2. Describe relevant interface features.
3. Answer the 20 accessibility questions.
4. Review conclusions, recommendations, the forward trace, and a backward proof.

Native-only platform questions are shown for desktop and mobile applications and hidden for web interfaces. Native results include a WCAG2ICT guidance link.

5. Inference and Explanation

5.1 Forward chaining

Forward chaining starts with all user observations and repeatedly applies inference clauses until no new fact can be added.

1. The engine verifies that an interface type is known.
2. It collects the base feature gates for each criterion.
3. For desktop or mobile software, it adds any native gates.
4. If all required features are Yes, it derives `applicable(Criterion)`.
5. If a required feature is No, it derives `not_applicable(Criterion)`.
6. If an applicable check has a No observation, the matching issue rule derives `issue(Criterion)`.
7. An issue derives `recommendation(Criterion)`.

The fixed-point process stops when no new conclusion can be produced.

5.2 Forward-chaining example

User input:

```
Interface type = Desktop application
Interactive functions = Yes
Platform keyboard service = Yes
Keyboard operation = No
```

Reasoning:

```
obs(interface_type, desktop)
+ obs(interactive_controls, yes)
+ obs(keyboard_service, yes)
-> applicable(c211)

applicable(c211)
+ obs(keyboard_operable, no)
-> rule r09
-> issue(c211)
-> recommendation(c211)
```

Final conclusion:

```
Possible issue: WCAG 2.1.1 Keyboard
Suggested action: Provide keyboard operation for every applicable function.
```

5.3 Backward chaining

Backward chaining begins with a selected goal, for example `issue(c211)`. The engine searches for the rule that can prove the goal. It then attempts to prove the rule premises: the criterion must be applicable and `keyboard_operable` must be No. Applicability becomes a subgoal requiring the interface type, interactive-controls fact, and, for native software, the platform keyboard-service fact.

If every premise is supported, the proof returns `provable: true`, the rule identifier, and a list of evidence. If a premise is missing, the issue is not proved. This does not automatically mean that the interface passes; it may still need review or may be not applicable.

5.4 Explanation facility

The explanation follows the required sequence:

```
User Input -> Observation Facts -> Rules Applied -> Reasoning -> Final Conclusion
```

For each result, the UI provides:

- the conclusion status;
- the rule identifier for a possible issue;
- the relevant observations;
- a plain-language explanation;
- a suggested action; and
- official WCAG and WCAG2ICT references.

Unknown answers are never reported as passes. This avoids a false conclusion when the user has not completed a check.

6. Testing

6.1 Test method

The project uses SWI-Prolog PlUnit. The suite contains 18 test definitions. One test iterates through 20 independent fixtures, so every issue rule from r01 to r20 is directly exercised. Other tests cover uncertainty, passing answers, non-applicability, multiple findings, native platform gates, forward trace order, backward proof, and source data.

Test command:

```
swipl -q -s tests.pl -g run_tests -t halt
```

6.2 Test cases and results

ID	Scenario and input	Expected output	Actual output	Verification
T01	Twenty individual failure fixtures	Each rule r01-r20 fires for its own criterion	Passed	Correct rule IDs returned.
T02	No interface type or answers	20 Needs review; zero passes	Passed	Unknown was not treated as Yes.
T03	Web; informative image = No; alt text = No	c111 Not applicable	Passed	Feature gate prevented r01.
T04	Web; informative image = Yes; alt text = Yes	c111 No issue reported	Passed	Applicable check accepted Yes.
T05	Three independent failing observations	Three possible issues	Passed	r01, r09, and r11 derived.
T06	Mobile; known correction; suggestion unsafe	c333 Not applicable	Passed	Safety exception prevented r20.
T07	Web; image present; alt text = No	Applicability before issue in trace	Passed	Trace order verified.
T08	Desktop; image present; alt text = No; prove c111	Backward proof succeeds using r01	Passed	Rule and evidence returned.
T09	Desktop; image absent; prove c111	Backward proof fails	Passed	Applicability could not be proved.
T10	Title answer = No; type omitted	Needs review	Passed	Type requirement enforced.
T11	Web; native title capability = No; title = No	r11 still fires	Passed	Native-only gate ignored for web.
T12	Desktop; title property = No; title = No	c242 Not applicable	Passed	Native title gate enforced.
T13	Mobile; title property = Yes; title = No	r11 fires	Passed	Native title rule applied.
T14	Mobile; keyboard service = No; keyboard operation = No	c211 Not applicable	Passed	Native keyboard gate enforced.
T15	Desktop; keyboard service = Yes; keyboard operation = No	r09 fires	Passed	Correct native rule triggered.
T16	Desktop; language exposure = No	Native guidance link present	Passed	WCAG2ICT link returned.
T17	Inspect fact and rule counts	50 facts and 20 issue rules	Passed	Required counts verified.
T18	Inspect criterion keys, numbers, and source fragments	Unique and present	Passed	Knowledge identifiers verified.

6.3 Test conclusion

All 18 automated test definitions passed. The web page, JavaScript, stylesheet, and JSON endpoints were also checked through the running local server. A desktop keyboard scenario returned r09 and displayed a successful backward proof with the interface type, feature gates, and failing observation.

7. User Manual

7.1 Required software

- SWI-Prolog 9.0 or later;
- a modern browser such as Firefox, Chrome, or Edge; and
- a terminal or PowerShell window.

The standard SWI-Prolog distribution contains the required HTTP, JSON, and PlUnit libraries. No database, web hosting service, Node.js package, or internet connection is required to run the adviser. Internet access is useful only for opening the W3C reference links.

7.2 Installation

1. Download SWI-Prolog from <https://www.swi-prolog.org/Download.html>.
2. Install it and allow the installer to add `swipl` to the system PATH when that option is offered.
3. Extract the project ZIP to any normal folder.
4. Open a terminal in the extracted project folder.

7.3 Starting the system

Windows PowerShell:

```
.\run.ps1
```

Git Bash, macOS, or Linux:

```
bash run.sh
```

Direct SWI-Prolog command:

```
swipl -q -s web_server.pl -- 8080
```

Open <http://localhost:8080/>. If port 8080 is occupied, choose another port, for example:

```
.\run.ps1 -Port 8081
bash run.sh 8081
```

Then open <http://localhost:8081/>. Press Ctrl+C in the terminal to stop the server.

7.4 Using the interface

1. Choose the interface type.
2. Inspect the target interface manually.
3. In **Describe the interface**, answer whether each feature is present.
4. In **Check what you observed**, answer each criterion question.
5. Use **Not sure** whenever the answer has not been verified.
6. Select **Show results**.
7. Read the status, explanation, action, and source link for each criterion.
8. To inspect backward reasoning, choose a criterion under **Ask why a criterion was flagged** and select **Check this goal**.
9. Expand **Show the forward-chaining rule trace** to view all derived conclusions.
10. Select **Clear answers** before reviewing another interface.

7.5 Example input and output

Example input:

Question	Answer
Interface type	Desktop application
Interactive functions	Yes
Platform keyboard service	Yes
2.1.1 Keyboard	No

Example output:

```
Status: Possible issue
Criterion: 2.1.1 Keyboard
Rule: r09
Reason: The criterion applies and keyboard operation was answered No.
Suggested action: Provide keyboard operation for every applicable function.
```

7.6 Running the tests

Windows PowerShell:

```
.\test.ps1
```

Git Bash, macOS, or Linux:

```
bash test.sh
```

A successful run reports 18 passed tests and returns exit code 0.

7.7 Troubleshooting

Problem	Solution
`swipl: command not found`	Install SWI-Prolog and restart the terminal, or use its full executable path.
Socket or port error	Start the system on a different port.
Browser cannot connect	Keep the terminal server running and check that the browser uses the same port.
Questions do not load	Confirm that all files remain in their original project folders.

8. Limitations and Future Work

The adviser covers 20 selected criteria rather than every accessibility requirement. Its answers depend on accurate human inspection, and short questions simplify detailed exceptions. It does not inspect source code, operate assistive technology, assess hardware, or issue legal or conformance decisions. Future work could add more criteria, platform-specific question sets, saved assessment reports, automated checks for objectively detectable conditions, and expert validation of the question wording.

9. Conclusion

The project demonstrates a complete small expert-system workflow. Reliable domain sources were converted into 50 Prolog facts and 20 explicit rules. The system supports forward chaining, backward chaining, explanation, a browser UI, and repeatable testing. It produces cautious preliminary conclusions while preserving uncertainty and applicability. The result is executable, explainable, and suitable for demonstrating knowledge engineering and rule-based reasoning.

References

[1] World Wide Web Consortium, "Introduction to Web Accessibility," W3C Web Accessibility Initiative, updated 3 February 2026. <https://www.w3.org/WAI/fundamentals/accessibility-intro/>

[2] World Wide Web Consortium, *Web Content Accessibility Guidelines (WCAG) 2.2*, W3C Recommendation, 12 December 2024. <https://www.w3.org/TR/WCAG22/>

[3] World Wide Web Consortium, *Guidance on Applying WCAG 2 to Non-Web Information and Communications Technologies (WCAG2ICT)*, W3C Group Note. <https://www.w3.org/TR/wcag2ict-22/>

[4] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[5] I. Bratko, *Prolog Programming for Artificial Intelligence*, 4th ed. Addison-Wesley, 2012.

[6] SWI-Prolog, "PlUnit: Unit Testing," SWI-Prolog Reference Documentation. <https://www.swi-prolog.org/pldoc/doc/_SWI_/library/ext/plunit/index.html>

Appendix A. Requirement Coverage

Assignment requirement	Report section / implementation evidence
Clearly defined domain	Section 2
Specialization, problem, decisions	Sections 2.2-2.4
Reliable domain and expert-system sources	Sections 2.5 and References
Knowledge engineering process	Section 3.1
At least 20 facts	Section 3.4: 50 facts
At least 20 rules	Section 3.5: 20 explicit rules
Explain conversion to facts and rules	Section 3.2
Prolog implementation	Section 4
Forward chaining	Section 5.1 and 5.2
Backward chaining	Section 5.3
User interface	Sections 4.4 and 7.4
Executable system and instructions	Section 7
Reasoning and explanation	Section 5
Test cases, expected and actual output	Section 6
User manual	Section 7